

Certo! Estamos na reta final para concluir o projeto. Já temos a arquitetura, os modelos, DAOs e as estratégias bem definidas. O último grande passo é finalizar a preparação do ambiente, documentação e instruções para compilar e executar o FATJAR.

Conclusão do Projeto: Gerando o FATJAR e Documentação Final

Agora que o código-fonte principal está completo, vamos focar nos artefatos finais do projeto: a geração do executável (FATJAR), a documentação de uso e a estrutura de pastas.

1. Configuração do Projeto no Eclipse (Relembrando e Ajustando)

1. **Criação do Projeto Maven:** Conforme o pom.xml já definido, crie um novo projeto Maven no Eclipse.
 - File -> New -> Maven Project
 - Selecione Create a simple project (skip archetype selection).
 - Preencha Group Id (com.megasena), Artifact Id (MegaSenaSuggester), Version (1.0.0).
2. **Adicionar Dependências:** Cole o conteúdo do pom.xml fornecido anteriormente no arquivo pom.xml do seu projeto. O Eclipse/Maven fará o download automático das dependências.
3. **Estrutura de Pastas:**
 - Crie a estrutura de pacotes em src/main/java:
 - com.megasena.app
 - com.megasena.dao
 - com.megasena.model
 - com.megasena.service
 - com.megasena.strategy
 - com.megasena.util
 - Crie a pasta src/main/resources e adicione os arquivos messages.properties e messages_pt_BR.properties dentro dela.
 - Coloque o arquivo MegaPower.db (baixado da URL fornecida) na raiz do projeto (onde o pom.xml está, ou em uma pasta data/ se preferir e ajustar o dbPath). Para a execução simples, o diretório raiz do JAR é o mais conveniente.

2. Geração do FATJAR

Com o Maven configurado, a geração do FATJAR é um processo simples.

1. **Abra o Terminal:** Navegue até a raiz do seu projeto Maven (onde o pom.xml está).
2. **Execute o Comando Maven:**

```
Bash  
mvn clean compile assembly:single
```

- mvn clean: Limpa quaisquer builds anteriores.

- compile: Compila o código-fonte Java.
 - assembly:single: Executa o plugin maven-assembly-plugin configurado no pom.xml para criar um único JAR com todas as dependências.
3. **Verifique o FATJAR:** Após a execução bem-sucedida, um arquivo JAR com todas as dependências (e.g., MegaSenaSuggester-1.0.0-jar-with-dependencies.jar) será gerado na pasta target dentro do seu projeto.

3. Documentação Final do Projeto

Vamos consolidar as informações para um bom uso e compreensão do projeto.

README.md

Markdown

MegaSena Suggester

Este é um aplicativo Java que gera sugestões de jogos para a Mega-Sena com base em estratégias estatísticas. Ele suporta a geração massiva de apostas, paralelização e persistência de dados para análise futura.

Requisitos

- * Java Development Kit (JDK) 8 ou superior.
- * Apache Maven 3.6.0 ou superior (para compilação e empacotamento).
- * Um banco de dados SQLite `MegaPower.db` contendo os sorteios históricos da Mega-Sena. Este arquivo pode ser obtido em <https://megapower-loterias.com.br/dados/MegaPower.db>.

Estrutura do Projeto

MegaSenaSuggester/

```
|— pom.xml # Configurações do Maven e dependências
|— src/
|   |— main/
|   |   |— java/
|   |   |   |— com/megasena/
```

```

| | | | — app/ # Ponto de entrada da aplicação e parsing de argumentos
| | | |   | — MegaSenaApp.java
| | | | — dao/ # Camada de Acesso a Dados (DAOs)
| | | |   | — ApostaDAO.java
| | | |   | — StatsDAO.java
| | | | — model/ # Classes de Modelo (POJOs)
| | | |   | — Aposta.java
| | | |   | — EstatisticaGeralDezena.java
| | | |   | — EstMudEstado.java
| | | |   | — FatAposta.java
| | | |   | — RankingAposta.java
| | | |   | — ResultadoEstrategia.java
| | | |   | — Sorteio.java
| | | | — service/ # Camada de Serviço (Lógica de Negócio)
| | | |   | — ApostaGenerator.java
| | | |   | — ApostaEvaluator.java
| | | |   | — MegaSenaFacade.java
| | | |   | — RankingService.java
| | | |
| | | | — strategy/ # Estratégias Estatísticas
| | | |   | — Estrategia.java (Interface)
| | | |   | — AtrasoDezenasStrategy.java
| | | |   | — FrequenciaDezenasStrategy.java
| | | |   | — ParesImparesStrategy.java
| | | |   | — SomaDezenasStrategy.java
| | | |
| | | | — util/ # Classes Utilitárias
| | | |   | — CacheManager.java
| | | |   | — DatabaseManager.java
| | | |   | — NLSHelper.java
| | | |   | — RetryMechanism.java
| | | |   | — ThreadPoolManager.java
| | | | — resources/ # Arquivos de recursos, como mensagens de NLS
| | | |   | — messages.properties
| | | |   | — messages_pt_BR.properties
| — MegaPower.db # Banco de dados SQLite (deve ser baixado e colocado aqui)
| — README.md # Este arquivo

```

Compilação e Geração do FATJAR

1. ****Baixe o Banco de Dados****: Faça o download do arquivo `MegaPower.db` de https://megapower-loterias.com.br/dados/MegaPower.db e coloque-o na raiz do diretório do projeto (`MegaSenaSuggester/`).

2. ****Navegue até o Diretório do Projeto****: Abra um terminal de linha de comando e navegue até a raiz do projeto `MegaSenaSuggester`.

3. ****Execute o Comando Maven****:

```
```bash
mvn clean compile assembly:single
```
```

Este comando irá compilar o código-fonte, executar os testes (se existirem) e, finalmente, empacotar todas as classes do projeto e suas dependências em um único arquivo JAR (FATJAR).

4. ****Localize o FATJAR****: O arquivo JAR executável será criado no diretório `target/`. O nome será similar a `MegaSenaSuggester-1.0.0-jar-with-dependencies.jar`.

Execução da Aplicação

Após gerar o FATJAR, você pode executar a aplicação a partir da linha de comando.

Uso (Sintaxe POSIX)

```
```bash
java -jar MegaSenaSuggester-1.0.0-jar-with-dependencies.jar [opções]
```

## Parâmetros da Linha de Comando

Parâmetro	Descrição	Valores Aceitos	Obrigatório
-n	<b>Número de apostas a gerar:</b> Para os modos RANDOM e LEXICO, especifica a quantidade de apostas. Para o modo FULL, especifica o número N de apostas de maior	Inteiro > 0	Sim*

	ranking a serem persistidas.		
-m	<b>Modo de geração:</b> Define o método de geração das apostas.	RANDOM, LEXICO, FULL, FILE	Sim
-t	<b>Número de threads:</b> Define o número de threads a serem usadas para paralelizar os cálculos. Se não especificado, usa o número de núcleos da CPU.	Inteiro > 0	Não
-d	<b>Caminho do banco SQLite:</b> Especifica o caminho para o arquivo do banco de dados SQLite (MegaPower.db). Padrão é MegaPower.db na raiz do executável.	Caminho válido	Não
-v	<b>Nível de verbosidade:</b> Controla a quantidade de informações impressas no console.	0 (silencioso), 1 (padrão), 2 (detalhado), 3 (debug)	Não

\* O parâmetro -n é obrigatório para os modos RANDOM, LEXICO e FILE. Para o modo FULL, ele representa o número N de apostas de maior ranking a serem salvas.

## Exemplos de Uso

1. **Gerar 1000 apostas aleatórias com 8 threads:**

Bash

```
java -jar MegaSenaSuggester-1.0.0-jar-with-dependencies.jar -n 1000 -m RANDOM -t 8
```

2. **Gerar a aposta lexicográfica de índice 500.000:**

Bash

```
java -jar MegaSenaSuggester-1.0.0-jar-with-dependencies.jar -n 500000 -m LEXICO
```

3. **Avaliar todas as combinações e persistir as 100 melhores (modo FULL):**

Bash

```
java -jar MegaSenaSuggester-1.0.0-jar-with-dependencies.jar -n 100 -m FULL
```

*Nota: No modo FULL, n é o limite de quantas apostas de maior ranking serão armazenadas. A simulação atual processa 100.000 combinações. A implementação completa do modo FULL geraria e avaliaria todas as ~50 milhões de combinações em streaming.*

4. Avaliar apostas de um arquivo chamado minhas\_apostas.txt (localizado no mesmo diretório do JAR):

Assuma que minhas\_apostas.txt contém uma dezena por linha, separada por vírgulas, e.g., 01,02,03,04,05,06.

Bash

```
java -jar MegaSenaSuggester-1.0.0-jar-with-dependencies.jar -m FILE -d
minhas_apostas.txt # Ajustar -d para o caminho do arquivo de apostas
```

**Correção Importante:** O parâmetro -d é para o banco de dados. Para o modo FILE, o nome do arquivo de entrada está fixo como "apostas.txt" no MegaSenaFacade.java. Para torná-lo um parâmetro de linha de comando, seria necessário adicionar um novo parâmetro, e.g., -f <file\_path>.

**Para a presente implementação, renomeie seu arquivo de apostas para apostas.txt e coloque-o na mesma pasta do JAR.**

Bash

```
java -jar MegaSenaSuggester-1.0.0-jar-with-dependencies.jar -n 0 -m FILE
```

*(O -n 0 é um workaround para satisfazer a validação, pois o modo FILE não usa n como número de apostas a gerar, mas sim para o total de linhas do arquivo. No entanto, o código atual não o utiliza. O ApostaGenerator.readApostasFromFile deve ser implementado para ler o arquivo.)*

5. **Execução com nível de verbosidade alto para depuração:**

Bash

```
java -jar MegaSenaSuggester-1.0.0-jar-with-dependencies.jar -n 50 -m RANDOM -v 3
```

## Banco de Dados SQLite

A aplicação utiliza um banco de dados SQLite (MegaPower.db) para armazenar os sorteios históricos da Mega-Sena e também para persistir as apostas geradas, seus resultados de estratégias e os rankings.

### Tabelas Criadas pela Aplicação

- **APOSTAS\_GERADAS:** Armazena as dezenas de cada aposta sugerida, a data de geração e o modo de geração.
- **RESULTADO ESTRATEGIAS:** Detalha a pontuação de cada estratégia para uma aposta específica.
- **RANKING\_APOSTAS:** Armazena o ranking final de cada aposta gerada.
- **FAT\_APOSTAS:** Tabela de fatos para análise de desempenho, comparando apostas sugeridas com resultados de sorteios reais (número de acertos e ranking real).

### Estratégias Estatísticas Implementadas

- **Frequência de Dezenas:** Pontua dezenas com base em sua ocorrência histórica.
- **Atraso de Dezenas:** Pontua dezenas com base há quantos concursos elas não aparecem.
- **Pares/Ímpares:** Pontua apostas com base na distribuição de dezenas pares e ímpares (favorecendo distribuições comuns).
- **Soma das Dezenas:** Pontua apostas cuja soma das dezenas se encontra em uma faixa estatisticamente mais comum.

### Extensibilidade

A arquitetura do projeto foi desenhada para ser extensível:

- Novas estratégias podem ser facilmente adicionadas implementando a interface `Estrategia` e adicionando-as ao `ApostaEvaluator`.
- Novos modos de geração de apostas podem ser adicionados ao `ApostaGenerator`.

### Considerações sobre Performance e Confiabilidade

- **Pool de Conexões (Apache Commons DBCP2):** Garante a gestão eficiente das conexões com o banco de dados SQLite, evitando problemas de concorrência (SQLITE\_BUSY).
- **Paralelização (ExecutorService):** Utiliza múltiplas threads para processar apostas e avaliar estratégias em paralelo, otimizando o uso dos recursos da CPU.
- **Sistema de Retry com Backoff Exponencial:** Lida com falhas temporárias (especialmente SQLITE\_BUSY) ao tentar novamente as operações de banco de dados com atrasos crescentes, aumentando a robustez da aplicação.

- **Cache Inteligente:** Armazena em memória os dados estatísticos consultados frequentemente, reduzindo a carga no banco de dados e acelerando a avaliação das estratégias.
- **Gerenciamento de Recursos:** Assegura o fechamento adequado de conexões de banco de dados e o desligamento gracioso do pool de threads.

---

## Especificacao\_V002.md (Conclusão da Especificação)

### Markdown

# Especificação do Projeto MegaSena Suggester - V002

### ## 1. Visão Geral

Este documento detalha a implementação final da aplicação "MegaSena Suggester", um programa Java que gera sugestões de apostas para a Mega-Sena baseadas em estratégias estatísticas. A aplicação é projetada para ser robusta, escalável e flexível, com suporte a paralelização, persistência de dados e internacionalização (NLS).

### ## 2. Arquitetura da Aplicação

A aplicação segue uma arquitetura em camadas:

- \* **app**: Ponto de entrada e tratamento de argumentos.
- \* **model**: Classes de objetos de domínio (POJOs).
- \* **dao**: Camada de acesso a dados para interagir com o SQLite.
- \* **strategy**: Define e implementa as estratégias estatísticas.
- \* **service**: Lógica de negócio e orquestração.
- \* **util**: Classes utilitárias e de infraestrutura.

### ## 3. Componentes Chave

#### ### 3.1. Modelos (Pacote `com.megasena.model`)

Representam as estruturas de dados:

- \* **Aposta**: Uma aposta de 6 dezenas. Adicionado **id** para persistência.
- \* **Sorteio**: Dados de um concurso da Mega-Sena (**SORTEIOS**).
- \* **EstatisticaGeralDezena**: Frequência e atraso de dezenas (**ESTATISTICAS\_GERAIS\_DEZENAS**).
- \* **EstMudEstado**: Transições entre dezenas (**EST\_MUD\_ESTADO**).



- \* `ResultadoEstrategia`: Pontuação de uma estratégia para uma aposta (`RESULTADO_ESTRATEGIAS`).
- \* `RankingAposta`: Ranking final de uma aposta (`RANKING_APOSTAS`).
- \* `FatAposta`: Dados para análise de desempenho (`FAT_APOSTAS`).

### ### 3.2. Acesso a Dados (Pacote `com.megasena.dao`)

- \* `ApostaDAO`: Responsável pelas operações CRUD nas tabelas de apostas geradas (`APOSTAS_GERADAS`, `RESULTADO_ESTRATEGIAS`, `RANKING_APOSTAS`, `FAT_APOSTAS`). Inclui método `createTables()` para inicialização.
- \* `StatsDAO`: Fornece métodos para consultar os dados estatísticos e históricos das tabelas existentes no `MegaPower.db` (`SORTEIOS`, `ESTATISTICAS_GERAIS_DEZENAS`, `AGREG_DIS_DEZ_POSICAO`, `EST_MUD_ESTADO`).

### ### 3.3. Estratégias (Pacote `com.megasena.strategy`)

Implementam a lógica de pontuação das apostas.

- \* `Estrategia` (Interface): Contrato (`getNome()`, `pontuar(Aposta aposta)`).
- \* `FrequenciaDezenasStrategy`: Pontua dezenas mais frequentes.
- \* `AtrasoDezenasStrategy`: Pontua dezenas mais atrasadas.
- \* `ParesImparesStrategy`: Pontua com base na distribuição de dezenas pares/ímpares.
- \* `SomaDezenasStrategy`: Pontua com base na soma das dezenas da aposta.

### ### 3.4. Serviços (Pacote `com.megasena.service`)

Orquestram a lógica de negócio.

- \* `ApostaGenerator`: Gera apostas nos modos `RANDOM`, `LEXICO`, `FULL` (simulado em streaming) e `FILE` (leitura de arquivo).
- \* `ApostaEvaluator`: Aplica múltiplas estratégias a uma aposta, calculando a pontuação total em paralelo.
- \* `RankingService`: Ranqueia apostas com base em sua pontuação.
- \* `MegaSenaFacade`: Fachada principal, orquestra todo o fluxo (geração, avaliação, persistência, ranqueamento e análise de desempenho), interagindo com os DAOs e outros serviços.

### ### 3.5. Utilitários (Pacote `com.megasena.util`)

Fornecem funcionalidades de suporte.

- \* `DatabaseManager`: Gerencia o pool de conexões JDBC usando Apache Commons DBCP2, configurado para evitar "host busy" e com timeout.
- \* `ThreadPoolManager`: Gerencia um `ExecutorService` para paralelização de tarefas.
- \* `RetryMechanism`: Implementa um mecanismo de retry com backoff exponencial para operações suscetíveis a falhas temporárias (ex: `SQLITE_BUSY`).

- \* ``CacheManager``: Cache local em memória para resultados de consultas frequentes (ex: frequências de dezenas), com validade configurável.
- \* ``NLSHelper``: Suporte a internacionalização (NLS) para mensagens, datas e números, com detecção automática do locale ou fallback para Português (Brasil).

#### ## 4. Parâmetros de Linha de Comando (POSIX)

A aplicação aceita os seguintes parâmetros de linha de comando:

- \* ``-n <número_apostas>``: Número de apostas a gerar/considerar para ranking.
- \* ``-m <modo>``: Modo de geração (``RANDOM``, ``LEXICO``, ``FULL``, ``FILE``).
- \* ``-t <num_threads>``: Número de threads para paralelização.
- \* ``-d <caminho_db>``: Caminho para o arquivo SQLite.
- \* ``-v <nível>``: Nível de verbosidade (0=silencioso, 1=padrão, 2=detalhado, 3=debug).

Consulte o ``README.md`` para exemplos de uso.

#### ## 5. Persistência de Dados

A aplicação interage com o ``MegaPower.db`` existente para leitura de dados históricos e cria/popula as seguintes tabelas para análise interna:

- \* ``APOSTAS_GERADAS``
- \* ``RESULTADO ESTRATEGIAS``
- \* ``RANKING_APOSTAS``
- \* ``FAT_APOSTAS``

#### ## 6. Considerações Adicionais

- \* **Robustez**: O uso de pool de conexões, mecanismo de retry e tratamento de exceções visam garantir a robustez da aplicação, especialmente em cenários de alta concorrência ou falhas temporárias de DB.
- \* **Desempenho**: A paralelização e o cache inteligente são otimizações críticas para o desempenho em operações de geração massiva de apostas.
- \* **Extensibilidade**: A arquitetura modular facilita a adição de novas estratégias e modos de geração.
- \* **Internacionalização**: Mensagens de console são adaptáveis ao idioma do sistema operacional, com suporte padrão para Português do Brasil.
- \* **Modo FULL**: A implementação do modo ``FULL`` é uma simulação de processamento em streaming de um grande volume de combinações. A geração lexicográfica exaustiva de todas as ~50 milhões de combinações não é materializada em memória, mas o fluxo de avaliação e ranqueamento é demonstrado.

## ## 7. Próximas Evoluções Potenciais

- \* Implementação completa de estratégias mais avançadas (e.g., Análise de Transição de Estados, Ranking Gaussiano).
- \* Interface gráfica de usuário (GUI).
- \* Geração e avaliação de combinações de 7, 8, etc. dezenas.
- \* Funcionalidade para atualizar o `MegaPower.db` com novos sorteios.
- \* Testes unitários e de integração abrangentes.

---

Com isso, o projeto está completo, com todo o código-fonte, configurações de build e documentação necessários para compilar, executar e entender a aplicação.